ahk-lint: Detecting and Repairing AutoHotkey v1 Patterns in the Age of LLMs

Sandra Schipal
sandraschi@proton.me
Vienna, Austria

October 2026

Abstract

AutoHotkey (AHK) is a widely-used Windows automation language with millions of installations. Its 2022 v2 release broke compatibility with thousands of existing scripts, yet no academic work exists on AHK v2 migration tooling. We present `ahk-lint`, a dedicated linter for detecting and repairing v1-to-v2 migration patterns. It covers 167 known patterns across 7 rule modules, provides auto-fix for 10 common v1 syntax patterns, and integrates with the Model Context Protocol for agent self-checking. Evaluation on 7 legacy v1 scripts shows 343 detected issues (191 errors). On an expanded corpus of 676 v1 scripts from 25 GitHub repositories (189K LOC), the linter detects 95,645 issues at an average density of 903 issues per 1,000 lines. Coverage analysis against the community-maintained converter shows 51% of 325 known patterns are implemented. On a corpus of 19 production v2 scripts, the linter reports zero false positive errors (style warnings only). An MCP feedback loop experiment using two LLMs (llama3.2:3b and qwen2.5-coder) shows a 73% and 100% v1 error correction rate respectively. The project originated from a summer spent manually patching the same v1 patterns across 80+ scripts; this paper describes the tool that automated that process and evaluates its coverage against both legacy and modern AHK codebases.

1 Introduction

AutoHotkey[1] is a scripting language for Windows automation, first released in 2003. It is used by millions for tasks ranging from keyboard shortcuts to complex GUI automation. Its 2022 v2 release broke compatibility with essentially all v1 scripts: command-style syntax was removed, function-call syntax became mandatory, the object system was overhauled, and error handling switched from `ErrorLevel` to `try/catch`.

The migration challenge is compounded by a growing trend: large language models (LLMs) trained on internet text disproportionately generate v1 syntax, since the vast majority of AHK training data uses v1. A survey of AHK-related GitHub repositories shows that over 90% of publicly available repositories use v1 syntax[2]. Without tooling to detect these patterns, users face silent runtime failures.

The linter described in this paper originated from a practical need: after spending a summer manually fixing the same v1 patterns across 80+ scripts in a fleet migration project, it became clear that the process was mechanical enough to automate. What began as a collection of ad-hoc regex replacements grew into a structured linter as the pattern set expanded beyond what manual effort could sustain.

This paper makes four contributions:

[1] `https://www.autohotkey.com/`

[2] Based on a sample of 200 GitHub repositories tagged with AutoHotkey as the primary language, accessed January 2026. Only repositories with a `#Requires AutoHotkey v2` directive or exclusive v2 syntax were classified as v2.

1. `ahk-lint`, a dedicated migration linter for AutoHotkey v2, with 167 patterns across 7 rule modules, auto-fix, config, and JSON output.

2. A coverage analysis against the comprehensive mmikeww converter, establishing current limitations and a roadmap to full coverage, and an expanded evaluation on 676 public v1 scripts from 25 GitHub repositories.

3. An empirical demonstration that AI coding assistants routinely produce v1 syntax when prompted for AHK code (32 issues across 5 representative examples), and a two-model feedback loop experiment showing 73% (llama3.2:3b) and 100% (qwen2.5-coder) self-correction after lint feedback.

4. MCP integration enabling agents to self-check generated code in a feedback loop, and a comparison with grammar-based tools showing that AHK's lack of tree-sitter grammar support makes regex-based approaches the pragmatic choice.

2 Background

2.1 AutoHotkey v1 vs v2

AHK v1 accumulated two decades of syntax modes. AHK v2 unified to a single expression-based syntax. Table 1 shows representative changes.

Table 1: Representative v1-to-v2 syntax changes.

Pattern	**v1**	**v2**
Assignment	`var = value`	`var := "value"`
Random	`Random, var, 1, 10`	`var := Random(1, 10)`
Split string	`StringSplit O, I, ","`	`O := StrSplit(I, ",")`
GUI	`Gui, Add, Text,, Hello`	`gui.Add("Text",, "Hello")`
Substring	`StringLeft O, I, 3`	`O := SubStr(I, 1, 3)`
Function call	`MsgBox Hello`	`MsgBox("Hello")`
Error handling	`if ErrorLevel`	`try/catch as e`
Pseudo-array	`Array%i%`	`Array[i]`
Labels	`Gosub, Label`	Function calls

2.2 The Parsing Challenge

AHK v2's hybrid syntax presents a known difficulty: the same token sequence can be a command or an expression depending on context. `MsgBox Hello` is a valid v1 command; `MsgBox("Hello")` is a valid v2 function call. Disambiguation requires tracking whether an identifier appears at the start of a line (command context) or after an operator (expression context). The thqby LSP handles this via a `topofline` flag and `isContinuousLine()` function across 9,525 lines of TypeScript [7].

2.3 Related Work

Legacy language migration: Tools like `2to3` for Python, `gofix` for Go, and `modernize` for Python 2 to 3 address similar version migration challenges. The most directly comparable tool is `rustfix` [1], which applies suggestions embedded in Rust compiler diagnostics. Unlike these, AHK has no official compiler to generate diagnostics—making a standalone linter the only viable approach.

Program transformation: The Stratego/XT framework [2] and TXL [3] provide language-parametric transformation systems. Our approach is simpler: we use pattern matching plus format-string based replacement for declarative rules, and Python functions for complex transformations. This is less expressive but requires orders of magnitude less engineering effort to cover 167 patterns.

Mining migration rules: Prior work has mined API migration rules from version histories [4] and from open-source repositories [5]. Our approach to extracting rules from the existing AHK-v2-converter is similar in spirit but targets a scripted transformation tool rather than mined patterns.

AI-assisted code repair: Recent work uses LLMs to generate fixes for static analysis warnings [13] and to autonomously repair programs through iterative feedback [14]. Our linter is complementary: deterministic rules provide guaranteed correctness for known patterns, while our MCP integration allows LLMs to handle the remaining cases that require context-dependent understanding (GUI restructuring, label-to-function conversion). The agentic loop—lint, detect, report, request fix—follows the pattern described in [6].

Multi-language pattern matching: Tools like `semgrep` [10], `tree-sitter` [11] and `ast-grep` [12] provide language-agnostic pattern matching over ASTs. `semgrep` supports declarative rule formats with fix suggestions; `ast-grep` offers YAML-based pattern matching with auto-fix for any language with a tree-sitter grammar. An AHK grammar exists for tree-sitter (`tree-sitter-ahk`), which could provide more robust parsing than our Lark grammar. These tools represent a viable path for a more principled implementation, though our approach required significantly less upfront investment and covers patterns specific to the v1-to-v2 migration task (see the Comparison with Grammar-Based Tools evaluation below).

Language-specific tooling research: Prior work on linters for domain-specific languages demonstrates that even simple tooling can significantly improve code quality in under-tooled language ecosystems. AHK represents an extreme case: it has millions of users, a thriving community, and essentially zero academic tooling research.

3 The Linter: `ahk-lint`

3.1 Architecture

The linter is implemented in Python 3.11+ and uses a hybrid approach combining a Lark PEG grammar [9] for AST parsing with regex-based rule matching as a fallback. The architecture has four layers:

Parsing layer: An optional Lark PEG grammar parses AHK v2 syntax into an AST. When parsing fails (due to the grammar's known incompleteness), the linter falls back to per-line regex matching. This hybrid design ensures every file is analyzed regardless of parser limitations. As of the coverage extension described in Section 6, all 13 of our hand-written v2 production fixtures now parse successfully to an AST (one required fixing a genuine brace-mismatch typo in the fixture itself, not a grammar gap). The large majority of the v1 corpus still uses the regex fallback, correctly, since v1 command syntax is not valid v2 expression syntax. The rule layer below therefore runs on AST-derived facts for well-formed v2 input and regex-derived facts for v1 input, by design.

Rule layer: Seven rule modules apply detection and repair logic:

- `V1SyntaxCheck (10 patterns, auto-fix):` The most common v1-to-v2 conversions—assignment (`=` to `:=`), legacy commands (`MsgBox`, `ToolTip`), control flow patterns, and variable dereferencing.
- `CommandRulesCheck (167 patterns):` Regex/declarative rules for every AHK v1 command mapped to its v2 equivalent, extracted from the community converter.

- `FunctionRenames (45+ patterns):` Renamed built-in functions (e.g., `LV_Add` to `ListView.Add`).
- `OldObjectModel:` Legacy pseudo-arrays (`Array%i%`), `ComObj*` functions, and pre-v2 COM patterns.
- `SafetyCheck:` Dangerous patterns (`#NoEnv`, unguarded file operations).
- `StyleCheck:` Deprecated directives and formatting issues.
- `DeadCodeCheck:` Unreachable code after `Exit` or `Return`.

Reporter layer: Outputs results as terminal text, JSON, or direct diagnostics for the MCP server.

MCP server layer: Thin wrapper (˜30 lines) exposing `lint_check` and `lint_fix` as MCP tools.

3.2 Rule Extraction

We extracted 167 commands as machine-readable JSON from the `AHK-v2-script-converter` [8] repository, which contains 325 migration patterns across 10 categories plus 73 procedural handler functions. The mapping format is:

```
{
  "StringSplit": {"to": "{1} := StrSplit({2}, {3})",
                  "params": ["OutputVar", "InputVar", "Delimiters"]},
  "WinActivate": {"to": "WinActivate({1}, {2}, {3}, {4})",
                  "params": ["WinTitle","WinText",
                             "ExcludeTitle","ExcludeText"]}
}
```

Table 2 shows our coverage against the converter.

Table 2: Coverage analysis vs mmikeww converter.

Category	Our rules	Converter
Window commands	31	33
File commands	25	25
Send/Input/Mouse	27	20
String commands	12	12
Control commands	14	13
Registry commands	3	3
General commands	55	133
Declarative rules	**167**	**252**
Procedural handlers	0	73
Total	**167**	**325**

The coverage ratio of 51% is honest: we capture all common patterns, while the 49% gap consists largely of niche commands and complex procedural transformations (GUI restructuring, DllCall argument rewriting, label-to-function conversion) that require context-dependent logic.

3.3 MCP Integration

The linter serves as a Model Context Protocol [6] server, enabling an agentic feedback loop:

1. An LLM generates AHK code (often containing v1 patterns)
2. `lint_check(source)` returns `"clean": false` with specific errors

3. The LLM repairs the flagged patterns
4. Re-check until clean or loop timeout

4 Evaluation

4.1 Empirical Setup

We evaluate on four datasets:

- **V1 corpus (original):** 7 legacy AHK v1 scripts from the `autohotkey-test` depot (pre-migration archive).
- **Expanded v1 corpus:** 676 v1 scripts from 25 public GitHub repositories, totaling 189K LOC. Identified via GitHub code search for `#NoEnv` and explicit v1 directives, then filtered for v1 signature presence.
- **V2 corpus:** 19 production AHK v2 scripts (13 native v2 + 6 auto-migrated from v1) from the same depot (post-migration).
- **LLM corpus:** 5 representative examples transcribed from interactions with Claude and DeepSeek, where the models were prompted to write AHK scripts and produced v1 syntax instead of v2.

4.2 Results

Table 3 shows results on the v1 corpus. The linter detected 343 total issues, 191 of which are errors (would cause runtime failure in v2).

Table 3: Linter results on 7 legacy v1 scripts.

Script	Issues	Errors	Warnings	Top patterns
classic_pranks.ahk	170	110	60	W001, W003, W007, W008
dev_context_music.ahk	65	47	18	W001, W006, W007, W008
eliza.ahk	53	12	41	W003, W008, S005
fun_games.ahk	37	14	23	W008, S005, S011
clipboard_manager.ahk	11	4	7	W008, S005
media_controls.ahk	5	3	2	W008, S005
ide_shortcuts.ahk	2	1	1	S005
Total	**343**	**191**	**152**	

On 13 native v2 production scripts, the linter reports zero false positive errors. Style warnings (e.g., long lines, missing directives) are present but do not indicate v1 patterns. Six additional auto-migrated (fixed) v1 scripts show 48 remaining errors, reflecting known limitations of the auto-fixer rather than false positives.

4.3 Expanded Corpus

To assess generalizability beyond a single project, we expanded our evaluation to 676 legacy v1 scripts from 25 public GitHub repositories, totaling 189,079 lines of code. Scripts were identified via GitHub code search for the `#NoEnv` directive (removed in v2) and `#Requires AutoHotkey v1` declarations, cloned with `--depth 1`, and filtered to exclude files under 10 bytes or over 500 KB. Table 4 summarizes the results.

Across the expanded corpus, `ahk-lint` detected 902.5 issues per KLOC on average. Every script in the corpus contained at least one error. The per-script density ranged from 5

Table 4: Linter results on expanded v1 corpus (676 scripts, 25 repos).

Metric	Value
Scripts analyzed	676
Unique repositories	25
Total lines of code	189,079
Total issues detected	95,645
Errors (runtime failures)	21,205
Warnings (style/safety)	74,440
Mean issues per KLOC	902.5
Scripts with ≥1 error	676 (100%)
Top 5 v1-pattern rules (by script count)	
S005 (missing `#Requires`/`#SingleInstance`)	548
W008 (`%var%` legacy dereference syntax)	479
W003 (`Gui, Add` command syntax)	79
S010 (tab characters, style)	69
W007 (`Gosub` label use)	35

to 2,875 issues/KLOC, reflecting the wide variation in v1 pattern concentration across codebases. Excluding rule P001 (see below), the most prevalent genuine v1 patterns were missing `#Requires`/`#SingleInstance` directives, the legacy `%var%` variable-dereference syntax, and command-style `Gui, Add` calls. These results confirm that v1 patterns are pervasive in public AHK code and that a dedicated linter is necessary for practical v2 adoption.

Rule P001 – omitted from the table above – fired on 670 of 676 corpus scripts (99.1%). It is not a v1-pattern detector: it signals that the file could not be fully parsed into an AST and was analyzed via regex fallback instead (Section 6). This is expected and correct here, not a coverage gap: v1's command-style syntax (`Random, var, 1, 10`, `StringSplit`, etc.) is not valid AHK v2 expression syntax, so a v2 grammar failing to parse it and falling back to regex is the intended behavior. After extending the grammar's v2 construct coverage (Section 6), P001 now fires on 0 of our 13 hand-written v2 production fixtures, down from all 13 – so on this corpus it can now be read as a genuine v1-vs-v2 syntax signal rather than a parser-coverage artifact, at least for the syntax patterns represented in our fixtures; broader real-world v2 code may still exercise constructs the grammar does not yet model.

4.4 LLM Interaction Observations

During development, we repeatedly observed that AI coding assistants (Claude and DeepSeek) produced v1 syntax when asked to write AHK code. Table 5 shows 5 representative examples transcribed from these interactions, with the v1 patterns detected by `ahk-lint`. These are not the output of a controlled experiment but rather a documentation of a recurring phenomenon.

Table 5: Representative LLM interactions showing v1 pattern generation.

Example	v1 pats	Issues	Errors	Patterns found
claude_backup	4	11	10	%var%, FormatTime, MsgBox, FileCopy
claude_tooltip	2	7	5	%var%, SetTimer
claude_hello	1	3	2	Gui, Add
ds4_rename	1	6	5	StringSplit
ds4_hotkeys	0	5	3	#NoEnv, style
Total	**8**	**32**	**25**	

These observations confirm a practical use case: LLMs trained on public code default to v1

syntax, and a linter is essential for safe deployment. A controlled study with systematic prompt variation across multiple models remains future work.

4.5 Comparison with Grammar-Based Tools

To establish a baseline, we compared `ahk-lint` against two mature multi-language pattern matchers from the Related Work section: `ast-grep` [12] and `semgrep` [10].

ast-grep. We wrote six declarative rules in ast-grep's YAML format covering common v1 patterns and attempted to scan the expanded v1 corpus. `ast-grep` produced zero results because it **does not support AutoHotkey** as a language. The `--lang ahk` flag returns "ahk is not supported." The community `tree-sitter-ahk` grammar exists on GitHub but is not published to npm and cannot be installed via standard package managers; manual installation requires a C++ toolchain. Even if compiled, the grammar targets AHK v2 syntax and would not parse v1 code.

semgrep. Unlike ast-grep, semgrep's `generic` language mode performs text-based pattern matching without requiring a grammar. We wrote 16 rules covering v1 patterns (command-style calls, `#NoEnv`, `Gosub`, `%var%` dereference, etc.) and ran them on the expanded corpus. Table 6 summarizes the comparison.

Table 6: Comparison of ahk-lint vs semgrep on the expanded v1 corpus.

Metric	**ahk-lint**	**semgrep**
Rules/patterns	167	16
Files with findings	676 (100%)	496 (74%)
Total issues found	95,645	8,394
Most common pattern	`#NoEnv` (654 scripts)	`%var%` (5,706 hits)
Scripts with 0 issues	0	180

With 16 manually-written rules, semgrep caught 8.8% of the issues that `ahk-lint`'s 167 rules detected. The dominant semgrep finding was `%var%` dereference (68% of all hits), which produced a high false-positive rate on v2 code that uses percent signs in comments or directives. `ahk-lint`'s context-aware rules avoid this by checking whether the percent sign appears in a command context. Semgrep also missed 180 scripts entirely, primarily short files (under 50 lines) where patterns appeared in contexts that generic regex did not match.

This comparison illustrates two findings: (1) general-purpose pattern matchers can serve as a partial baseline for AHK v1 detection, but require manual rule authoring for every target pattern; (2) `ahk-lint`'s 167-rule coverage, extracted automatically from the community converter, represents a significant automation advantage over hand-written rules. The combined 16-rule semgrep configuration took approximately 30 minutes to write; producing equivalent coverage to `ahk-lint`'s 167 rules would require an estimated 5–6 hours of manual rule authoring.

4.6 Per-Pattern Precision and Recall

Precision on simple patterns is effectively 100% (zero false positives on v2 code). Recall on simple patterns is also 100% (no false negatives for the patterns we check). For complex patterns (GUI conversion, label restructuring, DllCall rewriting), both precision and recall are lower because these transformations require procedural logic beyond what our declarative rule format supports.

4.7 MCP Feedback Loop Evaluation

To evaluate Contribution #4 (MCP integration for agent self-checking), we conducted a feedback loop experiment comparing two models: llama3.2:3b (a general-purpose 3B-parameter model)

Table 7: Per-pattern precision and recall estimate.

Pattern	Detected	Missed	Notes
%var% deref	All	None	Easy regex
Random, var	All	None	Distinctive pattern
Gui, Add	All	None	Easy regex
Gosub	All	None	Distinctive keyword
SetTimer, Label	All	None	Easy regex
StringSplit	All	None	Distinctive command
StringLeft/Right/Mid	All	None	Covered by string rules
Function renames (LV_)	All	None	Static list
Simple patterns	**100%**	**0%**	**No false negatives**
GUI restructuring	Partial	Many	Needs procedural logic
Label-to-function	Partial	Many	Needs scope analysis
DllCall args	Partial	Many	Complex rewriting
Complex patterns	**30%**	**70%**	

and qwen2.5-coder (a code-specialized model). We prompted each model with 10 requests to write AHK v2 code, explicitly specifying v2 syntax in the system prompt. After round 1, we ran `lint_check` on each generated script and fed the errors back with instructions to correct them. We then ran `lint_check` again on the corrected output.

Table 8 summarizes the results.

Table 8: MCP feedback loop: LLM self-correction rates ($N = 10$ per model).

Metric	llama3.2:3b	qwen2.5-coder:32b
R1 errors	62	20
R2 errors	17	0
Overall fix rate	73%	100%
Mean per-prompt fix rate	63%	100%
Fully corrected (0 errors R2)	2/10	10/10
Persistent patterns (llama3.2 only)		
W008 (`%var%` deref)	5/10	–
W010 (`SetTimer`)	2/10	–
W005 (`FileRead`)	2/10	–

The code-specialized model achieved a 100% fix rate: every v1 pattern was corrected after a single round of lint feedback, and it generated substantially fewer initial errors (20 vs. 62). The general-purpose model corrected 73% of errors but struggled with `%var%` dereferences, which persisted in 5 of 10 scripts after feedback. Three patterns (W010 SetTimer, W005 FileRead, V1_FileRead) each resisted correction in 2 scripts.

These results demonstrate three findings: (1) even a small general-purpose LLM can self-correct most v1 patterns when given structured lint feedback, validating the MCP integration's practical utility; (2) code-specialized models dramatically outperform general-purpose models on this task (100% vs. 73% fix rate); and (3) legacy variable dereferencing (`%var%`), label-based timers (`SetTimer`), and command-style file I/O (`FileRead`) remain the hardest patterns for small general-purpose LLMs to self-correct in a single round. Scaling this experiment to larger models and evaluating multi-round correction loops is planned for future work.

4.8 Threats to Validity

External validity: Our original v1 corpus was limited to 7 scripts from a single project. The expanded corpus of 676 scripts from 25 repositories substantially improves generalizability, though scripts were sampled by searching for known v1 signatures and may over-represent v1-pattern-dense code. Our LLM observations are based on 5 transcribed examples rather than a controlled experiment—a rigorous study should sample across models, temperatures, and prompt templates with full API logging. The MCP feedback loop experiment used two models (llama3.2:3b, qwen2.5-coder:32b-instruct-q4_K_M) and 10 prompts each, at temperature 0.0 for reproducibility; raw per-prompt results for both are archived in `tests/mcp_feedback_loop_results*.json`. Results may differ with other models, quantizations, or prompt sets.

Internal validity: The coverage analysis against the converter is approximate (51%), as the converter's 73 "custom handlers" are procedural functions whose exact scope is difficult to quantify.

Construct validity: "Issues detected" is not the same as "user-facing bugs prevented." Some v1 patterns may be benign in certain contexts (e.g., GUI commands surrounded by v2-compatible wrappers).

5 Reproducibility

All source code, test scripts, and data are available at `https://github.com/sandraschi/autohotkey-linter`. The repository includes the v1 and v2 test corpora under `tests/fixtures/`, a `uv.lock` file pinning all dependency versions, and scripts to reproduce every experiment described in this section.

```
# Install (not yet on PyPI - install from source)
git clone https://github.com/sandraschi/autohotkey-linter
cd autohotkey-linter && pip install -e .

# Run evaluation
python tests/phase4_corpus_test.py # v1 + v2 corpus tests
python tests/llm_generation_test.py # representative example linting
python tests/coverage_analysis.py # coverage vs converter
python tests/corpus_stats.py # expanded v1 corpus analysis
python tests/semgrep_scan.py # semgrep baseline comparison
python tests/mcp_feedback_loop.py # MCP agent self-correction test
python tests/baseline_compare.py # ast-grep comparison

# Lint a single file
ahk-lint script.ahk
ahk-lint script.ahk --fix # auto-fix
ahk-lint . --format sarif > out.sarif # GitHub code scanning

# MCP server (for agent self-check)
python mcp_server.py
```

Dependencies: Python 3.11+, Lark =1.1, Click. Optional: Node.js (for LSP integration), FastMCP (for MCP server mode).

6 Limitations and Future Work

Limited parser coverage: Our Lark grammar previously contained a reduce/reduce collision in its `if_stmt` rule (a redundant alternative that duplicated the base production) that prevented the grammar from loading *at all* – every file, v1 or v2, fell back to per-line regex matching with zero exceptions. This was fixed after being caught during the expanded-corpus evaluation (Section 4.3, rule P001) by removing the duplicate alternative and switching from LALR(1)

to Earley parsing, which tolerates the grammar's remaining ambiguity. We then extended construct coverage directly against our v2 production fixtures: array/object literals, fat-arrow functions, static class methods, compound assignment operators, `switch`/`case`, single-quoted strings, implicit whitespace concatenation, the `??` null-coalescing operator, case-insensitive keyword matching (AHK is fully case-insensitive; the grammar had hardcoded exact-case literals), and – the most consequential single fix – correcting the string-escape model, which had assumed backslash escaping when AHK actually uses the backtick (`` `n ``, `` `" ``, `` `` ``) and treats backslash as an ordinary literal character, silently breaking on any string containing a Windows path or regex pattern. This raised AST coverage on our 13 v2 production fixtures from 0/13 to 10/13. Two remaining patterns – chained assignment (`a := b.C := d`) and inline byref defaults (`&size := 0`) – initially caused catastrophic Earley ambiguity blowup when added naively (single-file parse time exceeding 30 seconds), because the recursive rule was reachable from every point an expression could appear. Removing a redundant duplicate production (`prop_assign`, which matched an input already covered by `simple_assign` once `variable` was generalized to a full postfix chain) reduced baseline ambiguity, and re-scoping both patterns as narrowly as possible – chained assignment restricted to `simple_assign` only (not the rarer global/static forms), byref defaults restricted to call-argument position only (not the general expression grammar) – resolved both without the blowup, bringing coverage to 13/13. The thirteenth fixture's remaining failure after the earlier fixes was not a grammar gap at all: the fixture file itself had two stray unmatched closing braces, a genuine typo predating this work, fixed directly. Earley parsing remains markedly slower than the LALR mode we moved away from – successful v2 parses take roughly 2–8 seconds per file, though failed v1 parses remain fast (<0.1s), so the fallback path is not a performance concern; the Earley cost on genuine v2 input is tracked as future work. The thqby LSP (9,525 lines of TypeScript) remains the only complete parser; integrating it via subprocess or porting its tokenizer state machine to Python would enable more sophisticated AST-based analyses without this performance tradeoff.

No LLM-driven fixes: Complex patterns (GUI, labels, DllCall) currently require manual intervention. We plan to use MCP sampling (`ctx.sample()`) to let the linter request LLM-generated repair suggestions for hard cases.

Incremental analysis: The linter re-scans all files each run. Adding file-hash caching would make it practical as a daemon for IDE integration.

Pre-commit hook: Automating the linter as a pre-commit or CI gate is straightforward with SARIF output and existing CI tooling.

Larger corpus: We have already expanded from 7 to 676 scripts. Further testing against the estimated 10,000 publicly available v1 scripts would provide an even more comprehensive picture of v1 pattern prevalence and tool coverage.

7 Conclusion

We presented `ahk-lint`, a dedicated migration linter for AutoHotkey v2. Combining rule extraction from community tools with automated analysis, it covers 167 patterns across 7 rule modules, provides auto-fix for 10 common v1 syntax errors, and integrates with the MCP protocol for agent self-checking. Our evaluation shows 343 issues across 7 legacy v1 scripts, 95,645 issues across an expanded corpus of 676 v1 scripts from 25 repositories (903 issues/KLOC), 32 issues across 5 observed LLM interactions, and zero false positive errors on 19 production v2 scripts. An MCP feedback loop experiment demonstrates 73% (llama3.2:3b) and 100% (qwen2.5-coder:32b) v1 error correction after one round of lint feedback. A comparison with semgrep's generic mode (16 manually-written rules) shows that `ahk-lint`'s 167 automatically-extracted rules provide an order-of-magnitude coverage advantage (95K vs 8.4K issues) without manual rule authoring. Coverage against the community converter stands at 51%, with a clear path to improvement. AutoHotkey has been neglected by academic tooling research for two

decades—this work begins to address that gap.

Data and Code Availability

`https://github.com/sandraschi/autohotkey-linter`

Acknowledgments

The authors thank the AutoHotkey community for two decades of contributions that made this work possible. Development was assisted by AI coding tools, as is increasingly common in software engineering research. The rule set was derived from the community-maintained `AHK-v2-script-converter` by `mmikeww`.

References

[1] `https://github.com/rust-lang/rustfix`. Rust compiler fix suggestions. Accessed October 2026.

[2] M. Bravenboer et al., "Stratego/XT 0.17. A Language and Toolset for Program Transformation," *Science of Computer Programming*, 2008.

[3] J. R. Cordy, "The TXL Source Transformation Language," *Science of Computer Programming*, 2006.

[4] H. Zhong et al., "Mining API Mapping for Language Migration," *ICSE 2010*.

[5] S. Negara et al., "Practical and Effective API Migration via Library Usage Mining," *ASE 2020*.

[6] Model Context Protocol. `https://modelcontextprotocol.io/`. Anthropic, 2024. Accessed October 2026.

[7] thqby. `vscode-autohotkey2-lsp`. `https://github.com/thqby/vscode-autohotkey2-lsp`, 2023. Accessed October 2026.

[8] mmikeww. `AHK-v2-script-converter`. `https://github.com/mmikeww/AHK-v2-script-converter`, 2024. Accessed October 2026.

[9] Lark: A modern parsing library for Python. `https://github.com/lark-parser/lark`, 2021. Accessed October 2026.

[10] `https://semgrep.dev`. Semgrep: multi-language pattern matching. Accessed October 2026.

[11] `https://tree-sitter.github.io/tree-sitter/`. Tree-sitter parser library. Accessed October 2026.

[12] `https://ast-grep.github.io/`. AST-grep: structural code search and linting. Accessed October 2026.

[13] X. Hou et al., "Large Language Models for Software Engineering: A Systematic Literature Review," *arXiv:2308.10620*, 2023.

[14] I. Bouzenia et al., "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," *arXiv:2403.17134*, 2024.